

Fredy Agent-Service

Architektur & Ablauf

`services/agent` + `packages/agent-core`

Grundlage für die Mikroarchitektur-Evaluation

Was der Agent-Service ist

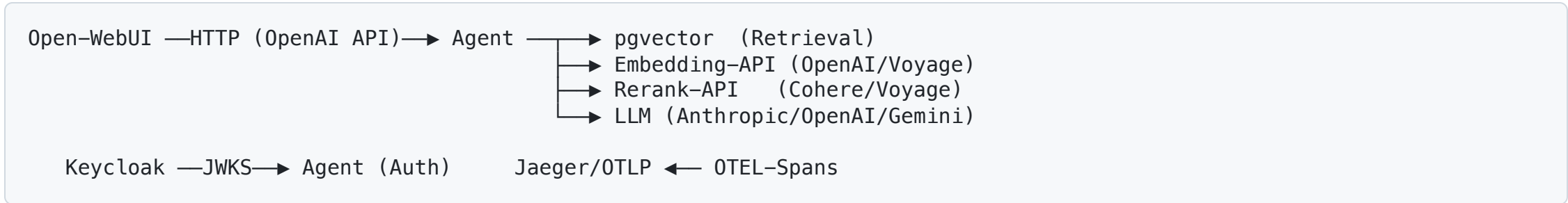
Ein **OpenAI-kompatibler HTTP-Endpoint** zwischen Open-WebUI und der Wissens-/LLM-Infrastruktur.

Kernaufgabe:

Chat-Anfrage nehmen → Kontext aus pgvector retrieven → LLM damit grounden → antworten.

Bei fehlendem Kontext: **bewusst verweigern** (Refusal statt Halluzination).

Rolle im Gesamtsystem



Drei Endpoints (`server.ts:67`):

- `GET /health` — unauthentifiziert
- `GET /v1/models` — registrierte **Agents = OpenAI-Modelle**
- `POST /v1/chat/completions` — Chat, Streaming (SSE) + Non-Streaming

Der Agent kennt kein OpenAI-Format — `openai/types.ts` ist reine Adapterschicht.

Zwei Schichten

`@fredy/agent-core` — agentagnostische Base

- `agents/agent.ts` — Contracts: `AgentDefinition`, `AgentRun`, `AgentRunInput/Result`
- `agent-registry.ts` — `id` → Agent-Dispatch
- `llm/resolve-chat-model.ts` — ChatModel-Factory (Präfix → Provider)
- `tools/tool-registry.ts` + `tools/rbac.ts` — Tool-Lookup + Rolle → Allowlist
- `otel/*`, `logging/*`, `config/define-config.ts`

`services/agent` — konkreter RAG-Agent + HTTP-Layer

- `main.ts` — **Composition Root** (Wiring)
- `server.ts` + `plugins/{auth,rbac,rate-limit}.ts` — Cross-Cutting via Hooks
- `routes/chat-completions.ts` — SSE via `reply.hijack()`
- `agents/rag-agent/*`, `tools/*`, `rerank/*`

Modul-Landkarte: agent-core

Modul	Verantwortung
agents/agent.ts	Zentrale Contracts (Agent, Run, Messages, Usage)
agent-registry.ts	Map id → Agent; Dispatch für Models + Chat
resolve-chat-model.ts	Modell-ID-Präfix → LangChain-Provider + Fallback
tool-registry.ts	Map Name → LangChain-Tool
tools/rbac.ts	Rolle → Tool-Allowlist (reine Funktionen)
otel/tracing.ts	NodeSDK-Bootstrap, idempotent
otel/langchain-callback.ts	LangChain-Runs → OTEL GenAI-Spans, Leak-Schutz
logging/logger.ts	pino-Logger-Factory

Modul-Landkarte: services/agent (1/2)

Bootstrap & Config

Datei	Verantwortung
main.ts	Composition Root — Pool, Profile, Store, Tools, Agent, Server
config.ts	loadConfig — Env→ AppConfig , Fail-Fast-Validierung
profile.ts	löst Tabelle + Embedding aus rag_profiles -Registry auf

Server & Plugins (Cross-Cutting via Fastify-Hooks)

Datei	Verantwortung
server.ts	Fastify-Assembly; Auth onRequest , RBAC+RateLimit preHandler
plugins/auth.ts	Keycloak-JWT (jose/JWKS) oder statischer API-Key
plugins/rbac.ts	Rolle auflösen → req.allowedToolNames
plugins/rate-limit.ts	TokenBucket per IP → 429 + Retry-After

Modul-Landkarte: services/agent (2/2)

Agent / Graph

Datei	Verantwortung
rag-agent.ts	AgentDefinition ; invoke / stream , SSE-Mapping, Span-Verwaltung
graph.ts	LangGraph StateGraph: retrieve / refuse / generate
retrieval.ts	Query-Split → vector_search → Pooling → optional Rerank
query-split.ts / token-utils.ts / system-prompt.ts	Multi-Query, Budget, Prompt

Tools & Rerank

Datei	Verantwortung
tools/vector-search.ts	embed → Store-Search → content_and_artifact
tools/pgvector.ts	Cosine-SQL 1 - (embedding <=> \$1) , sanitizeIdentifier
rerank/factory.ts + adapters	provider → Cohere/Voyage/null

Request-Flow — HTTP-Schicht

```
POST /v1/chat/completions
├── onRequest: Auth-Hook      (auth.ts)
│               Keycloak-JWT (jose/JWKS) ODER static API-Key (timingSafeEqual)
│               → req.jwtRole = extractRole(claims)
├── preHandler: RBAC-Hook     (rbac.ts)
│               resolveRole → req.allowedToolNames = filterToolsForRole(role)
├── preHandler: RateLimit     (rate-limit.ts)
│               TokenBucket per request.ip → 429+Retry-After | weiter
└── handler:    chat-completions.ts
│               Zod-Validierung → AgentRunInput → agents.get(model)
```

Request-Flow — Agent + Graph

```

RAG-Agent (rag-agent.ts)
  span "agent.run" → graph.streamEvents(state, { callbacks:[OTEL] })
  ↓
LangGraph StateGraph (graph.ts)
  START → retrieve
    retrieveContext: splitQueries → vector_search je Query
                    → Pooling (dedup) → optional Rerank(topN, ≥threshold)
                    → context: string | null
    |
    conditional edge: context === null ?
      └─ true → refuse → RAG_FALLBACK_RESPONSE (KEIN LLM-Call)
      └─ false → generate → trim → SystemPrompt+History → model.invoke
  → END
  ↓
mapRagStreamEvents → SSE-Frames data:{chunk}\n\n → [DONE] (reply.hijack)
  
```

Schlüsselstellen im Flow

- **Tools** werden in `main.ts` registriert; der Graph ruft sie **nicht direkt**, sondern über `retrieveContext` (`retrieval.ts:86`).
- **RAG-Profil / Tabelle** wird **einmalig beim Boot** aufgelöst (`main.ts:31`), nicht pro Request → `PgVectorStore` an feste Tabelle gebunden.
- **Reranking mit Threshold** (`retrieval.ts:148`): nur `score ≥ rerankThreshold` bleibt; leere Menge → `null` → **Refusal**. Rerank-Fehler → graceful Fallback auf unrerankten Kontext.
- **OTEL-Spans**: `agent.run` → `retrieval` / `rerank` → `gen_ai.chat` / `gen_ai.tool.execute`, korrekt genestet — auch im Streaming.

Der StateGraph im Detail

State-Shape (`RagStateAnnotation` , flaches Record):

```
Input:  sessionId, requestId, messages[], userMessage,  
        allowedToolNames?, temperature?, maxTokens?, startedAt  
Working: context: string|null, answer, usage?, responseModel?
```

Kein `messages` -Reducer → keine agentische Schleife.

Deterministische lineare Pipeline, einzige Bedingung: `context === null` .

```
START → retrieve —[context===null?]→ refuse → END  
                                     ↳ generate → END
```

`null` entsteht bei: Tool nicht verfügbar / RBAC-denied / keine Queries / leerer Retrieval / Rerank filtert alles weg.

Nodes

- **retrieve** (async) — reine Kontextbeschaffung, kein LLM → setzt `{ context }`
- **refuse** (sync) — `answer = RAG_FALLBACK_RESPONSE`, **kein LLM-Call**, LogEvent `finishReason: "fallback"`
- **generate** (async) — Token-Budget-Trim → System-Prompt + History → `model.invoke` → `usage/responseModel`

History-Handling (`toLangChainHistory`): System-Messages raus (Prompt wird selbst gebaut), user→Human / assistant→AI, aktuelle User-Message anhängen.

Retriever Kontext wird **nur in generate** in den System-Prompt injiziert — nie in die State-History.

Erweiterungspunkte — factory- & registry-getrieben

Zweiter Agent auf derselben Base:

1. Neue `AgentDefinition` implementieren (`createXAgent(): AgentDefinition`)
2. In `main.ts` registrieren: `agentRegistry.register(createXAgent(), deps)`
3. Sofort unter `/v1/models` + aufrufbar via `model: "x-agent"` — **keine HTTP-Änderung**

Ein Agent kann intern LangGraph, ReAct oder gar keinen Graph nutzen — der Vertrag ist nur `invoke` / `stream`.

Austausch der Bausteine

Baustein	Mechanismus	Konfiguration
ChatModel	<code>resolveChatModel</code> Präfix→Provider	<code>LLM_FALLBACK_MODEL</code> + Keys
Reranker	<code>Reranker</code> -Port + Factory- case	<code>RERANKER=none cohere voyage</code>
RAG-Profil	<code>rag_profiles</code> -DB-Registry	<code>RAG_PROFILE</code> (+Env-Fallback)
Tools	<code>ToolRegistry.register</code>	in <code>main.ts</code> , auto-RBAC-fähig
Embedding	<code>createEmbeddingClient</code>	provider + endpoint
Graph-Nodes	eigenständige Funktionen	<code>Partial<State></code> -Rückgabe

Test-Seams durchgängig: `verifyToken` , `createModel` , `fetchImpl` , `QueryablePool` .

Kopplung: saubere Grenzen

- **HTTP ↔ Agent** — nur über `AgentDefinition` (invoke/stream)
- **Reranker / Embedding** — echte Ports & Adapters
- **Auth / RBAC** — isolierter Hook / reine Funktionen
- **Config** — Fail-Fast am Boot, zentral
- **Composition Root** (`main.ts`) — Constructor-DI, kein Container, kein globaler State (außer OTEL-Singleton)

Agent → agent-core: nur über Barrel `index.ts` , keine Deep-Imports, Abhängigkeit strikt einseitig, **keine** Zyklen.

Kopplungs-Hotspots ⚠ — für die Evaluation

#	Hotspot	Bewertung
1	LangChain/LangGraph durchzieht Graph, Retrieval, Factory, SSE. <code>mapRagStreamEvents</code> hängt an internen Event-Namen	Framework-Lock-in, fragil bei Version-Bumps
2	RAG-Agent fest an <code>vector_search</code> -Tool + <code>artifact.hits</code> -Shape	Nicht wirklich tool-agnostisch
3	pgvector-Schema an Python-Importer gekoppelt, ohne geteilten Schema-Vertrag	Cross-Service-Datenkopplung
4	Boot-Zeit-Bindung von Profil/Tabelle	Kein per-Request / Multi-Tenant-Wechsel

Fazit

- **Grundmuster:** Modular Monolith — agentagnostische Base + konkreter Service, Abhängigkeit strikt einseitig, keine Zyklen
- **Stil intern:** Ports & Adapters an den Rändern, Registry-Pattern für Agents & Tools, Composition Root, Cross-Cutting via Hooks
- **RAG-Kern:** deterministischer LangGraph (`retrieve → refuse|generate`), bewusste Refusal-Strategie, Multi-Query + Rerank-mit-Threshold
- **Stärken:** schmale Contracts, durchgängige Test-Seams, Fail-Fast-Config, OTEL von Anfang an
- **Angriffspunkte:** 4 Kopplungs-Hotspots (LangChain-Tiefe, Tool-Bindung, Schema-Kopplung, Boot-Bindung)

Diskussion

Einstieg in die Mikroarchitektur-Evaluation:

Hotspots einzeln durchgehen — oder Bewertungsraster

(Kopplung/Kohäsion, Ersetzbarkeit, Service-Schnitt Richtung Repo-Splitting)?